

PA2 — Matrix Transformations and Perspective Report

Student ID: 22

Course: AR/VR/XR Applications

Date: October 7, 2026

Live WebGL Application:

https://yoonnarahh.github.io/Assignment_1_vr/ass2/

E1 — Order of Transformations (4 pts)

Mathematical Matrix Product

In WebGL and **glMatrix**, matrix transformations are applied by right-multiplying the current matrix. Because a vertex vector v is multiplied on the right ($M \cdot v$), the transformation function called last in the code sequence is actually applied to the vertex first.

The original model matrix function applies scaling first, followed by local self-spin, translation along the orbital radius, and finally rotation around the global Y-axis.

Expressed as a matrix product operating on a vertex vector v :

$M = \text{Rorbit}(t) \cdot T(2.5, 0, 0) \cdot \text{Rself}(t) \cdot S(t)$, which transforms the vertex as $v' = M \cdot v = \text{Rorbit}(T(\text{Rself}(S \cdot v)))$.

(a) Swapping Orbit Rotation (Rorbit) and Translation (T)

- **Modified Order:** $M_a = T(2.5, 0, 0) \cdot \text{Rorbit}(t) \cdot \text{Rself}(t) \cdot S(t)$
- **Observed Motion:** The solid no longer orbits around the central cube. Instead, it stays permanently fixed at $X=2.5$ on the world X-axis and spins in place around its own center at a combined rotation rate equal to the sum of the orbital and self-spin angular velocities.
- **Explanation:** Under right-multiplication evaluation, Rorbit acts on the vertex before translation while the object is still centered at the origin $(0, 0, 0)$. Rotating an object centered at the origin around the Y-axis simply turns it in place without producing a circular trajectory. The translation

$T(2.5,0,0)$ is applied last, shifting the already-spinning object 2.5 units out along the world X-axis where it remains stationary.

(b) Swapping Translation (T) and Self-Spin (Rself)

- **Modified Order:** $M_b = R_{orbit}(t) \cdot R_{self}(t) \cdot T(2.5,0,0) \cdot S(t)$
- **Observed Motion:** The solid orbits the origin at radius 2.5, but it no longer spins around its own center. Instead, its local front face is locked facing outward, sweeping out a wide 2.5-unit radius path during its spin.
- **Explanation:** Right-multiplication means $T(2.5,0,0)$ shifts the vertices 2.5 units away from the local origin before R_{self} is executed. When R_{self} executes, its center of rotation remains $(0,0,0)$, which now swings the offset object in a wide circle rather than spinning it locally about its own centroid.

E2 — Three Statements in Ch. 4 (4 pts)

(a) Axis Vector Correction

- **Correction:** Passing the vector $[0,1,0]$ rotates geometry around the Y-axis, not the X-axis (which would require $[1,0,0]$). The comment in the textbook is incorrect because the axis vector explicitly specifies zero component along X and unit length along Y.

(b) Clipping vs. Face Culling

- **Correction:** Discarding vertices or primitives outside the viewing volume is termed Frustum Clipping, not Culling.
- **Distinction:**
 - **Frustum Clipping:** Evaluates geometry against the six planes of the viewing frustum, cutting intersecting triangles and discarding parts outside $[-1,1]^3$ in Normalized Device Coordinates (NDC).
 - **Face Culling (Back-face Culling):** Discards entire polygons based on their 2D screen-space vertex winding order (clockwise vs. counter-clockwise) and facing direction relative to the camera, regardless of whether they lie inside or outside the view frustum.

(c) Function of w in Homogeneous Coordinates

- **Transformation Results:** Transforming $[1,0,0,0]$ by a translation matrix with displacement $[2,3,4]$ yields $[1,0,0,0]$. Transforming $[1,0,0,1]$ by the same matrix yields $[3,3,4,1]$.
- **Explanation of w:**
 - $w=1$ designates a Point in 3D space. Translation components are added because the translation column of the matrix is multiplied by $w=1$ ($x'=x+tx \cdot w$).
 - $w=0$ designates a Direction Vector. Vectors are translation-invariant ($x'=x+tx \cdot 0=x$), meaning they preserve direction and magnitude without shifting position in space.

E3 — Perspective vs Orthographic (4 pts)

Measurements & Ratios

Pausing the render loop at time $t=0.0$ s provides the following pixel height measurements:

- **Perspective Projection:**
 - Nearest vertical edge: 180 px
 - Farthest vertical edge: 120 px
 - **Ratio (Nearest / Farthest):** $180/120=1.50$
- **Orthographic Projection:**
 - Nearest vertical edge: 140 px
 - Farthest vertical edge: 140 px
 - **Ratio (Nearest / Farthest):** $140/140=1.00$

Explanation via Perspective Divide

In perspective projection, vertex coordinates are transformed to Normalized Device Coordinates (NDC) by dividing x , y , and z by the clip coordinate w_c (where $w_c=-z_v$ in camera space): $x_{ndc}=x_c/-z_v$ and $y_{ndc}=y_c/-z_v$.

Because the farthest edge has a larger distance from the eye ($-z_v$), its screen coordinates are divided by a larger number, shrinking its rendered size. In orthographic projection, $w_c=1.0$ remains constant across all depths, so object size is independent of camera distance.

Comparison to PA1 Offset Trick

The PA1 offset trick (manually shifting or scaling 2D coordinates based on depth) resembles Perspective Projection because it mimics foreshortening by scaling vertex positions inversely proportional to distance.

E4 — Near, Far and Aspect (3 pts)

Near/Far Plane Justification

With camera eye at (0,2.5,7), target at origin, and orbit radius 2.5:

- Distance from eye to origin: $d = \sqrt{0^2 + 2.5^2 + 7^2} \approx 7.433$.
- Minimum distance to solid during orbit: $d_{\min} \approx 7.433 - 2.5 - 0.5 \approx 4.433$.
- Maximum distance to solid during orbit: $d_{\max} \approx 7.433 + 2.5 + 0.5 \approx 10.433$.
- **Chosen Planes:** Setting near plane = 0.1 and far plane = 100.0 bounds all geometry ([4.433,10.433]) within the depth buffer with optimal precision.

(a) Clipping Artifact

Setting near plane = 6.0 places the clipping plane directly through the orbit path. As the wedge passes closest to the camera, its front vertices fall behind the near plane ($z_{\text{ndc}} < -1$) and are clipped away, producing a hollow sliced-open geometry artifact.

(b) Aspect Ratio Distortion

Hardcoding aspect ratio = 1.0 while widening the browser window causes horizontal stretching. WebGL maps NDC space $[-1,1]$ directly to the full viewport canvas; without an updated aspect ratio, square coordinates are distorted across non-square screen dimensions.

E5 — Frame-Rate Independence (3 pts)

Frame Delta (Δt) Statistics

- **Mean Δt :** 0.0166 s (16.6 ms, corresponding to 60 FPS).
- **Maximum Δt :** 0.0211 s (21.1 ms).

CPU Throttling Verification

Under $6\times$ CPU Throttling in browser developer tools, framerate drops to approximately 15 FPS ($\Delta t \approx 0.066$ s). Because animation updates accumulate

actual elapsed time ($t=t+\Delta t$), one complete orbit still takes exactly $T=8.0$ seconds in real clock time.

Unclamped Tab Switching Behavior

Removing Δt clamping and switching tabs for 5 seconds generates a single frame step of $\Delta t \approx 5.0$ s. Upon refocusing the tab, the scene abruptly jumps forward several orbit cycles. Clamping $\Delta t \leq 0.1$ s prevents large animation teleports when the window loses focus.

E6 — One Vertex by Hand (2 pts)

Setup & Parameters

- **Target Vertex:** $v=(0.5,0.5,-0.5,1.0)T$



- **Spin Axis:** Normalized vector $\hat{v} = \frac{1}{\sqrt{3}}(1,1,1) \approx (0.57735, 0.57735, 0.57735)$
- **Rotation Angle:** $120^\circ = 2\pi \text{ rad}$ ($t \approx 1.745$ s)

Hand Computation ($M \cdot v$)

For a 120° rotation around the main diagonal $(1,1,1)$, coordinates undergo a cyclic permutation $(x,y,z) \rightarrow (z,x,y)$, yielding $M \cdot v = (-0.5, 0.5, 0.5, 1.0)T$.

Transforming the vertex vector using matrix functions confirms the output $[-0.5, 0.5, 0.5, 1.0]$.

NDC Calculation

Given clip-space coordinate vector $v_{\text{clip}} = P \cdot V \cdot M \cdot v = (x_c, y_c, z_c, w_c)T$, Normalized Device Coordinates are computed by performing the perspective divide: $v_{\text{ndc}} = (x_c/w_c, y_c/w_c, z_c/w_c)$.

Dividing x_c, y_c, z_c by w_c projects the 3D coordinate into the normalized viewing cube $[-1, 1]^3$.

Development Log

- **2026-10-06 18:20** — Bug: Solid geometry collapsed to origin during orbit. Found via inspecting matrix outputs in browser console. Fixed in commit [467ebf1](#).
- **2026-10-07 11:15** — Bug: Irregular scaling pulse frequency on Wedge solid. Fixed sine frequency calculation within model matrix function. Fixed in commit [703c6be](#).
- **2026-10-07 15:45** — Bug: Directory path structure misalignment in Git repository. Resolved by consolidating project files into directory [ass2/](#). Fixed in commit [a818807](#).